

REPORTE DE LABORATORIO

Actividad: 17 de Junio de 2026

Arquitectura Multi-Nivel (N-Tier) y Patrón Lógico de Software (MVC) en .NET

Nombre completo:	Samuel Esteban Pichillá Duarte
Carné:	202505036
CUI:	3735407520101
Curso:	Introducción a la Programación y Computación 2
Sección:	Laboratorio A
Fecha de entrega:	17 de junio de 2026

Resumen: El presente reporte desarrolla el análisis teórico y la implementación práctica de la Arquitectura Multi-Nivel (N-Tier) y el Patrón MVC (Model-View-Controller) en el entorno .NET 8. Se abordan los conceptos de distribución física de componentes, desacoplamiento lógico de software, ciclo de vida de peticiones HTTP y enrutamiento semántico en ASP.NET Core, con el objetivo de comprender el diseño de sistemas escalables, seguros y mantenibles bajo los estándares de la industria.

I. PARTE 1: Fundamentación Teórica y Análisis Crítico

1.1 El Tránsito hacia los Sistemas Distribuidos y Multi-Capa

La Limitación del Monolito Local

En un sistema monolítico local, la interfaz de usuario, la lógica de negocio y el almacenamiento de datos coexisten en una única máquina física aislada. Este diseño impone serias restricciones sobre la **sincronización y escalabilidad**: cuando múltiples usuarios necesitan acceder simultáneamente a los datos, se generan conflictos de concurrencia, bloqueos (deadlocks) y condiciones de carrera que el sistema no puede resolver de forma distribuida. La escalabilidad se vuelve imposible, ya que el único mecanismo disponible es el escalamiento vertical (agregar más RAM o CPU a la misma máquina), con límites físicos y económicos muy claros. Además, una falla en cualquier componente (hardware, red local o software) detiene la operación completa del sistema, eliminando toda posibilidad de tolerancia a fallos. La actualización de cualquier módulo exige detener todo el sistema, lo que conlleva tiempos de inactividad inaceptables en entornos productivos modernos.

Distinción Crítica: Layers vs. Tiers

Esta distinción es fundamental en la arquitectura de software y frecuentemente se confunde:

Concepto	Definición	Naturaleza
Layer (Capa)	División lógica del código fuente. Representa cómo está organizado internamente el software en grupos de responsabilidades.	Organización virtual del código
Tier (Nivel)	División física del despliegue. Indica en qué máquina o servidor reside y ejecuta cada parte del sistema.	Separación física de infraestructura

La clave está en que un sistema puede tener tres capas lógicas (Presentación, Negocio, Datos) pero estar desplegado en un solo tier (monolito) o en tres tiers físicos separados. La confusión entre estos términos lleva a decisiones de infraestructura erróneas.

Responsabilidades en la Arquitectura de 3 Niveles

Nivel	Nombre	Misión exclusiva	Tecnologías comunes
Tier 1	Presentación	Interacción directa con el usuario. Renderizado de la interfaz gráfica. Captura de eventos y envío de peticiones al Tier 2.	HTML5, CSS3, JavaScript, React, Angular, Blazor, Android/iOS.
Tier 2	Aplicación / Negocio	Procesamiento de la lógica de negocio. Validación de reglas. Orquestación del flujo de datos entre Tier 1 y Tier 3.	ASP.NET Core, Node.js, Spring Boot, Django, PHP (Laravel).

Tier 3	Datos	Persistencia, consulta y gestión de datos. Garantía de integridad referencial y transaccionalidad ACID.	SQL Server, MySQL, PostgreSQL, MongoDB, Oracle DB, Redis.
---------------	-------	---	---

Seguridad Perimetral: Exposición de Puertos de Base de Datos

Exponer públicamente el puerto de una base de datos (por ejemplo, el puerto 3306 de MySQL o el 1433 de SQL Server) hacia internet constituye un **error crítico de seguridad** por múltiples razones de ingeniería:

- **Superficie de ataque masiva:** Cualquier agente en internet puede intentar ataques de fuerza bruta contra las credenciales del motor de base de datos.
- **Inyección SQL directa:** Sin la capa de aplicación como intermediario validador, un atacante puede enviar consultas SQL arbitrarias directamente al motor.
- **Exposición de metadatos:** Los motores de base de datos en sus versiones por defecto pueden revelar información sobre su versión, esquema y estructura de tablas.
- **Vulnerabilidades de día cero:** Los motores de bases de datos tienen historial de CVEs críticos que se explotan masivamente cuando el puerto está expuesto.

Buena práctica recomendada: El Tier 3 debe residir en una **red privada segmentada** (subred privada o VLAN) que solo acepte conexiones entrantes provenientes del Tier 2 (servidor de aplicaciones), mediante reglas de firewall que bloqueen todo tráfico no autorizado. Adicionalmente, se debe emplear cifrado TLS para las conexiones intra-tier, rotación periódica de credenciales y principio de mínimo privilegio en las cuentas de base de datos.

1.2 Desacoplamiento Lógico con el Patrón MVC

La Crisis del Código Espagueti

Mezclar sentencias SQL, lógica matemática y etiquetas visuales en un mismo archivo físico genera lo que la ingeniería denomina **código espagueti**. Los impactos negativos son:

- **Mantenimiento inviable:** Un cambio en la estructura de la base de datos obliga a revisar manualmente todos los archivos que contienen SQL embebido, multiplicando el riesgo de errores e inconsistencias.
- **Imposibilidad de pruebas unitarias:** No se puede probar la lógica de negocio de forma aislada si está mezclada con código de acceso a datos y con generación de HTML. Las pruebas unitarias requieren aislar una unidad de comportamiento, lo que es estructuralmente imposible con código acoplado.
- **Duplicación de código:** La misma lógica tiende a repetirse en múltiples archivos, violando el principio DRY (Don't Repeat Yourself).
- **Curva de onboarding elevada:** Un nuevo desarrollador debe entender simultáneamente SQL, lógica de negocio y tecnología de presentación para modificar una sola funcionalidad.
- **Acoplamiento tecnológico:** Cambiar el motor de base de datos o la tecnología de presentación requiere reescribir prácticamente todo el sistema.

Separación de Preocupaciones (SoC) — Componentes del Patrón MVC

El patrón MVC fue formulado originalmente por **Trygve Reenskaug** en 1979 en el laboratorio de Xerox PARC, y establece tres componentes con responsabilidades radicalmente aisladas:

Componente	Responsabilidad Central	Restricciones de Aislamiento
Modelo (Model)	Representa el estado, los datos y las reglas de negocio del dominio. Gestiona la persistencia y la lógica de dominio.	No debe conocer nada sobre cómo se muestran los datos. No contiene referencias a elementos visuales, ni a HTML, ni a objetos de vista. Su comportamiento es idéntico independientemente de si existe una interfaz gráfica o no.
Vista (View)	Presentar los datos al usuario de forma visual. Es una entidad pasiva que recibe datos ya procesados y los renderiza.	Tiene estrictamente PROHIBIDO contener: lógica de negocio, sentencias SQL, validaciones de dominio o cálculos matemáticos complejos. Solo puede contener lógica de presentación mínima (iteración sobre listas para mostrarlas). Se define como 'inteligente' porque puede adaptar su renderizado, pero 'pasiva' porque no toma decisiones de negocio.
Controlador (Controller)	Actúa como intermediario táctico y director de orquesta. Recibe las peticiones HTTP, coordina al Modelo y selecciona la Vista a retornar.	No debe contener lógica de negocio compleja. Su rol es: (1) recibir la petición, (2) invocar al Modelo para obtener/modificar datos, (3) preparar los datos para la Vista, (4) retornar la Vista apropiada. El antipatrón 'Fat Controller' ocurre cuando el controlador acumula responsabilidades que corresponden al Modelo.

Métricas de Ingeniería: Alta Cohesión y Bajo Acoplamiento

El patrón MVC contribuye directamente a dos métricas fundamentales de la ingeniería de software:

- **Alta Cohesión (High Cohesion):** Cada componente agrupa responsabilidades estrechamente relacionadas. El Modelo solo contiene lógica de datos, la Vista solo lógica de presentación y el Controlador solo lógica de enrutamiento. Esto significa que un cambio en los requisitos de presentación no afecta al Modelo, y un cambio en la lógica de negocio no afecta a la Vista.
- **Bajo Acoplamiento (Low Coupling):** Los tres componentes se comunican mediante interfaces bien definidas, no mediante dependencias directas de implementación. El Controlador conoce al Modelo y a la Vista, pero el Modelo no conoce a la Vista ni al Controlador. Esto permite sustituir una Vista (por ejemplo, cambiar de HTML a JSON para una API REST) sin modificar una sola línea del Modelo.

En términos cuantitativos, un sistema bien diseñado con MVC permite que el 70-80% de los cambios de requisitos impacten un solo componente, reduciendo el costo y riesgo de las modificaciones. Las métricas LCOM (Lack of Cohesion of Methods) y CBO (Coupling Between Objects) de sistemas MVC bien implementados son sistemáticamente mejores que los sistemas monolíticos sin separación de responsabilidades.

II. PARTE 2: Modelado del Ciclo de Vida y Enrutamiento Semántico

2.1 Mapeo Analítico de URLs — Plantilla de Enrutamiento ASP.NET Core

El motor de enrutamiento convencional de ASP.NET Core utiliza la siguiente plantilla jerárquica por defecto para mapear URLs a clases y métodos C#:

{controller=Home}/{action=Index}/{id?}

Donde: **{controller}** = nombre de la clase controladora (sin el sufijo "Controller"), **{action}** = nombre del método a ejecutar, **{id?}** = parámetro opcional. Los valores después del signo = son los valores por defecto cuando el segmento está ausente.

URL Entrante del Cliente	Clase Controladora	Método (Acción)	Parámetro id
/ControlAcademico/Login	ControlAcademicoController	Login	(Ninguno / Opcional)
/Estudiante/Historial/20260123	EstudianteController	Historial	20260123
/Asignacion/Detalle/10	AsignacionController	Detalle	10
/Home	HomeController	Index (por defecto)	(Ninguno / Opcional)

Nota: Las URLs mostradas corresponden a la ruta relativa después del dominio base. El motor de enrutamiento de ASP.NET Core es case-insensitive por convención.

2.2 Ciclo de Vida Completo de una Petición Web en MVC

A continuación se describe de forma secuencial el viaje completo que realiza una petición HTTP desde que el usuario interactúa con el navegador hasta que recibe el HTML renderizado:

Paso 1

El Navegador Genera la Petición HTTP

El usuario hace clic en un botón o enlace en su navegador web. El navegador construye una petición HTTP (GET o POST) con la URL destino, las cabeceras (headers) necesarias y, en caso de formularios, el cuerpo (body) con los datos. La petición viaja por internet hacia el servidor web. En este punto, el componente activo es exclusivamente el navegador del cliente; ningún componente del servidor ha intervenido aún.

Paso 2	El Motor de Enrutamiento Analiza la URL La petición arriba al servidor donde está alojada la aplicación ASP.NET Core. El middleware de enrutamiento (UseRouting) intercepta la petición y analiza la URL utilizando la plantilla configurada {controller}/{action}/{id?}. El framework identifica qué clase controladora y qué método de acción deben procesarla. Por ejemplo, /Estudiante/Listar mapea hacia EstudianteController.Listar(). El Controlador es instanciado por el contenedor de inyección de dependencias.
Paso 3	El Controlador Delega la Lógica al Modelo El método de acción del Controlador es ejecutado. Siguiendo el principio de Skinny Controller, el Controlador no procesa datos directamente, sino que invoca al Modelo para solicitar los datos que necesita. El Modelo ejecuta la lógica de negocio, accede a la fuente de datos (base de datos, servicio externo, caché), aplica las reglas de dominio y devuelve el resultado al Controlador. El Controlador recibe el resultado limpio del Modelo.
Paso 4	El Controlador Selecciona y Alimenta la Vista Con los datos ya obtenidos del Modelo, el Controlador llama a View(datos) especificando qué archivo de Vista debe renderizarse y pasando los datos como parámetro (ViewBag, ViewData o un objeto Model fuertemente tipado). La Vista recibe exclusivamente los datos necesarios para su presentación, sin haber tenido ningún contacto directo con el Modelo ni con la base de datos. En ASP.NET Core MVC, las Vistas son archivos .cshtml (Razor) ubicados en la carpeta Views/.
Paso 5	La Vista Genera el HTML y se Envía la Respuesta El motor de vistas Razor procesa el archivo .cshtml, combinando el HTML estático con las instrucciones C# para renderizar los datos dinámicamente. El resultado es un string de HTML puro. El framework lo encapsula en una respuesta HTTP con código de estado 200 OK, las cabeceras correspondientes (Content-Type: text/html) y el cuerpo HTML. La respuesta viaja de vuelta al navegador del usuario, que la interpreta y renderiza la página final que el usuario ve en pantalla. El ciclo ha concluido.

III. PARTE 3: Implementación Práctica — Sistema de Control Académico

Se construyó una aplicación web funcional bajo ASP.NET Core MVC en .NET 8, aplicando estrictamente la separación de responsabilidades y el patrón de Controladores Delgados (Skinny Controllers). La estructura del proyecto sigue la convención estándar de la industria:

Archivo / Directorio	Capa MVC	Responsabilidad
Models/Estudiante.cs	Modelo	Entidad POCO de dominio. Define la estructura de datos sin dependencias externas.
Controllers/EstudianteController.cs	Controlador	Despachador delgado. Recibe peticiones HTTP, delega al modelo y retorna vistas.
Views/Estudiante/Listar.cshtml	Vista	Template Razor para renderizar la lista de estudiantes en HTML.
Views/Shared/_Layout.cshtml	Vista (Layout)	Layout maestro reutilizado por todas las vistas de la aplicación.
Program.cs	Infraestructura	Punto de entrada. Registra servicios y configura el pipeline HTTP con enrutamiento.

3.1 Verificación del Desacoplamiento — Auditoría de Calidad

Prueba de Cohesión (GET /Estudiante/Listar)

Al acceder a la acción Listar mediante una petición GET, la respuesta refleja un diseño limpio y cohesionado: el Controlador se limita exclusivamente a extraer la colección de estudiantes del almacenamiento en memoria y pasarla a la Vista mediante `return View(_baseDatosMemoria)`. No contiene cálculos internos, sentencias SQL en texto plano, ni lógica de presentación. La Vista recibe el modelo tipado `IEnumerable<Estudiante>` y lo renderiza en HTML sin tomar ninguna decisión de negocio.

Evaluación de Antipatrones — Controladores Delgados

Ambos métodos del controlador (Listar y Registrar) cumplen la restricción de no superar las 20 líneas de código, evitando el antipatrón de Fat Controllers. La validación en el método Registrar es perimetral (solo verifica que los datos no sean nulos o inválidos) y delega toda responsabilidad adicional al modelo de datos. Este diseño garantiza que el controlador sea testeable de forma aislada y que los cambios de lógica de negocio futura se implementen en el Modelo, no en el Controlador.

Verificación de Endpoints HTTP

Endpoint	Verbo HTTP	Comportamiento Esperado	Código de Respuesta
----------	------------	-------------------------	---------------------

/Estudiante/Listar	GET	Retorna la vista HTML con la tabla de todos los estudiantes registrados.	200 OK
/Estudiante/Registrar	POST	Valida y agrega un nuevo estudiante. Retorna JSON con ubicación del recurso creado.	201 Created
/Estudiante/Registrar (datos inválidos)	POST	Detecta carne ≤ 0 o nombre vacío y rechaza la solicitud.	400 Bad Request

IV. PARTE 4: Conclusiones y Aprendizajes

1 Viabilidad de la Arquitectura N-Tier

La separación física en múltiples niveles resuelve los problemas de escalabilidad y tolerancia a fallos que son inherentes a los sistemas monolíticos locales. La posibilidad de escalar horizontalmente el Tier 2 (replicar servidores de aplicación) sin afectar al Tier 3 demuestra la flexibilidad operativa que este modelo arquitectónico proporciona.

2 El Patrón MVC como Estándar de la Industria

La implementación práctica confirma que MVC reduce significativamente el acoplamiento entre componentes. El hecho de que la Vista (Listar.cshtml) y el Controlador puedan modificarse de forma independiente sin romperse mutuamente evidencia que el patrón cumple su promesa de mantenibilidad a largo plazo.

3 Skinny Controllers como Disciplina de Diseño

La restricción de 20 líneas por método no es arbitraria: obliga al desarrollador a reflexionar sobre dónde pertenece cada línea de código. Si una lógica no cabe en 20 líneas en el controlador, es señal de que debe existir en el Modelo o en un servicio de dominio dedicado. Esta disciplina previene la deuda técnica desde el primer día.

4 Seguridad como Principio de Diseño Temprano

La justificación sobre la exposición de puertos de bases de datos demuestra que la seguridad no puede añadirse como una capa posterior, sino que debe ser un criterio fundamental en las decisiones de arquitectura desde las primeras etapas del diseño del sistema.

V. PARTE 5: Referencias Bibliográficas

Las siguientes fuentes primarias del laboratorio constituyen el fundamento normativo del presente reporte, conforme a los requisitos formales de entrega establecidos por el cuerpo docente:

Referencia Central 1 — Sesión 11

Facultad de Ingeniería, USAC. (2026). *Sesión 11: Modelado Base y Arquitecturas de Despliegue. Evolución de Sistemas Distribuidos, Fundamentos del Modelo Cliente-Servidor y Diseño Físico Multi-Capas (N-Tier)*. Laboratorio del curso Introducción a la Programación y Computación 2. Universidad de San Carlos de Guatemala.

Referencia Central 2 — Sesión 12

Facultad de Ingeniería, USAC. (2026). *Sesión 12: Arquitectura y Componentes del Patrón MVC. Desacoplamiento Lógico de Software, Ciclo de Vida de las Peticiones y Enrutamiento en Aplicaciones Interactivas Modernas*. Laboratorio del curso Introducción a la Programación y Computación 2. Universidad de San Carlos de Guatemala.

Referencia Complementaria 3

Microsoft Corporation. (2024). *ASP.NET Core MVC Overview*. Microsoft Learn. Recuperado de: <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>

Referencia Complementaria 4

Reenskaug, T. (1979). *Thing-Model-View-Editor: An Example from a Planning system*. Xerox PARC Technical Note. Palo Alto, California, EE.UU.